Internet of Things — Project Proposal

GreenWave EMS

Smart Green Wave Emergency Management System

Hybrid IoT Architecture for Smart Traffic Light Management in Smart Cities

Created by

G. Innocenti (5299405) **& A. Scarpulla** (5231799)

Academic Year 2025/2026

What is GreenWave EMS?

Problem

Traditional, static, and non-reactive intersections

During emergencies, passive road infrastructure forces ambulances to navigate critical obstacles:

- Blocking red lights
- Queued traffic
- Unaware crossing pedestrians

Solution

Intelligent and dynamic road infrastructure

GreenWave EMS transforms the road into an active and communicative system to ensure maximum safety:

- Real-time detection
- Temporary cycle variation
- Creation of a safe "green wave"

The Operational Sequence of GreenWave EMS

The Scenario

Intersection Criticality

Active Emergency: An ambulance must cross the city in the shortest possible time.

Static & dynamic obstacles:

- Blocking red lights
- Traffic and waiting queues
- Crossing pedestrians

Pre-emption

Sequential Logic

1

Detection & Transmission

Constant GPS telemetry sent via MQTT protocol.

2

Threshold Verification

Node-RED receives data and calculates the distance from the intersection.

3

Green Wave

Temporary activation and MQTT command to the intersection.

Actuation & Analysis

Edge Control & LLM Report

Local Security (Edge)

The ESP32 receives the command, validates local security and executes the physical unlock.

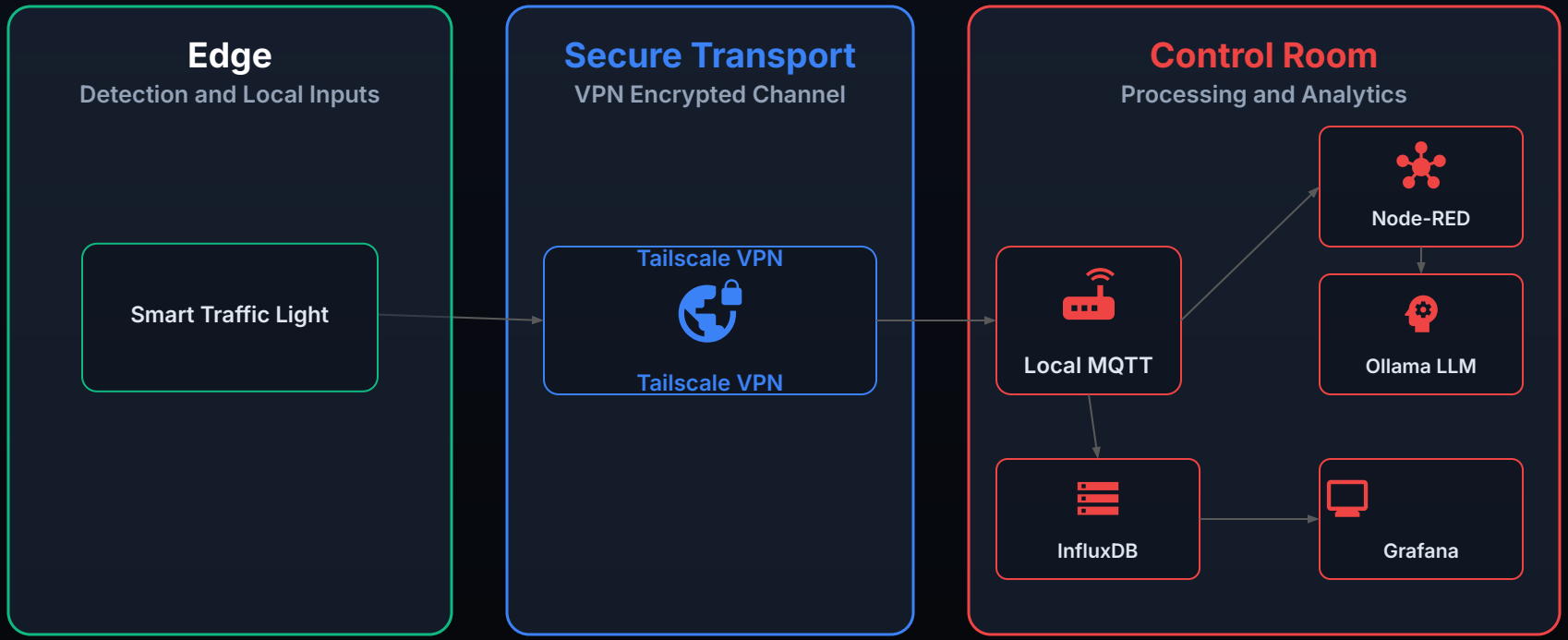
Recovery & Logging

Once the ambulance has passed, the intersection returns to normal and the event is logged.

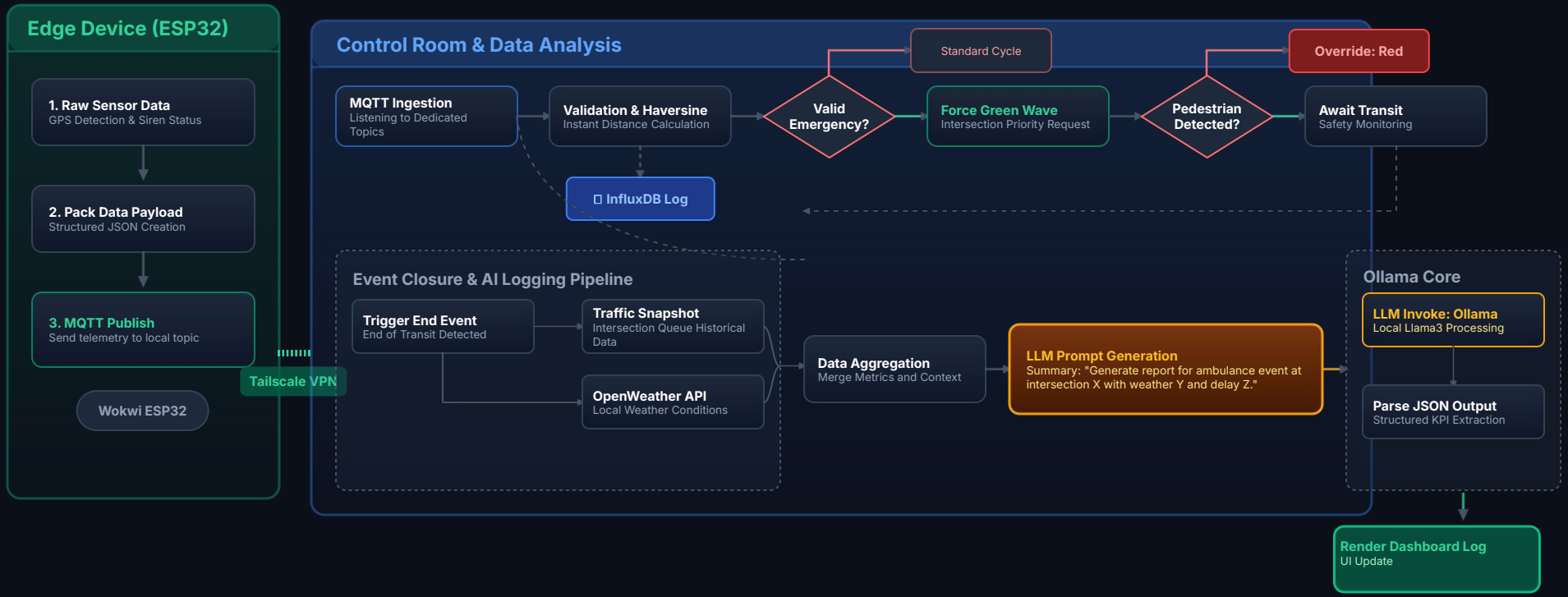
AI Report (LLM)

Intelligent post-event analysis and automatic generation of the operational report.

System Architecture



GreenWave EMS Flowchart



Telemetry Simulation & Node.js IoT

Ambulance Simulator

Node.js Simulation: Replaces physical hardware for dynamic acquisition of GPS and vehicle state.

Real-Time Telemetry: Reproduces the progressive approach to the intersection as a smart sensor.

Spatial Awareness (V2I): Active feedback loop. Listens to the traffic light status and forces an emergency brake (300m threshold) in case of pedestrians.

JSON MESSAGE EXAMPLE

Parameters are serialized into a structured payload ready for transmission:

```
{
"latitude": 44.40,
"longitude": 8.93,
"siren_active": true
}
```

1. Position calculation → 2. Telemetry → 3. JSON Serialization → 4. Publishing & Listening (V2I)

Pub/Sub Architecture & MQTT Communication Channels

The MQTT Network Protocol

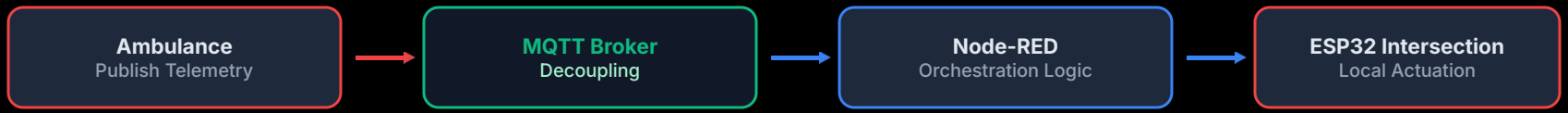
The choice of **MQTT (Message Queuing Telemetry Transport)** is consistent with the need to decouple data producers and consumers.

The system uses a centralized **MQTT Broker** as an intermediary, eliminating any coupled or direct connection between field devices and the backend.

Topic Segmentation

Using separate channels ensures modularity and security, isolating critical information flows:

- Telemetry: `city/ems/ambulance1/gps`
- Commands: `city/ems/trafficlight1/cmd`
- Feedback: `city/ems/trafficlight1/status`
- Traffic: `city/ems/trafficlight1/queue`



Architectural Advantage: The asynchronous structure **eliminates direct coupling** , making the entire smart city infrastructure highly scalable and maintainable.

Infrastructure Security and Isolation

The Problem: Cloud Simulator Limitations

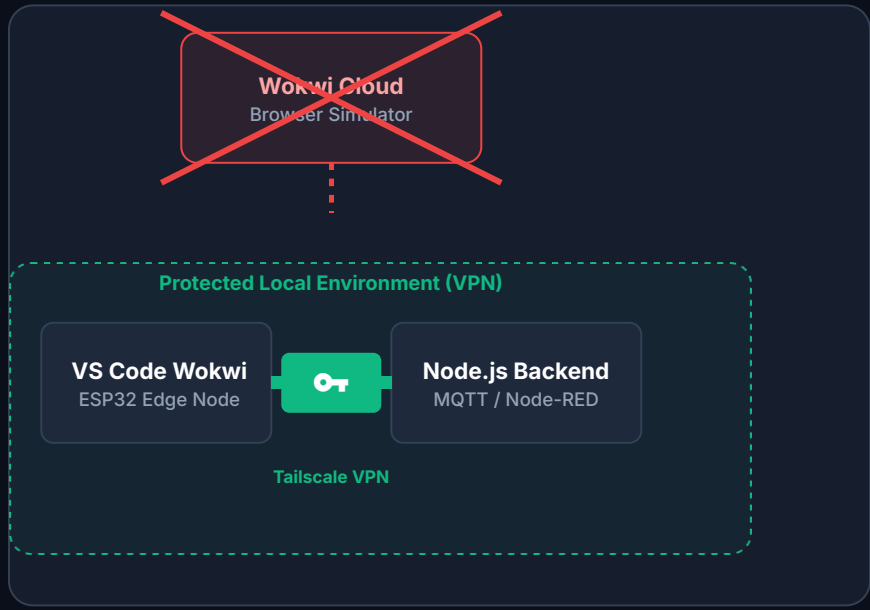
- **NAT Isolation:** Wokwi Cloud cannot communicate with the local backend behind routers.
- **Zero Security:** Public topics in plain text are interceptable and vulnerable.
- **High Latency:** Inefficient global routing (round-trip from third-party cloud servers) compared to direct peer-to-peer routing between nodes.

The Solution: Local (VS Code) + Tailscale

- Wokwi integrated in **VS Code** entirely locally.
- Edge ESP32 joined in the **Tailscale VPN Mesh** (IP 100.107.9.36).

The Architectural Result

Creation of a **private and encrypted end-to-end tunnel**. The traffic light connects to the Control Center, bypassing the public internet.



NETWORK IMPLEMENTATION (ZERO-TRUST VPN)

Distributed Network Infrastructure: Secure Tunneling via Tailscale

tailscale

Operational Challenge

Overcome router NAT obstacles to connect the Edge (Sicily) with the central server (Rapallo) without exposing data on the public internet.

VPN Implementation

Configuration of a peer-to-peer network via Tailscale to join the two development nodes into a single encrypted virtual LAN.

Backend Configuration (Rapallo)

Assignment of static private IP 100.107.9.36 to the Backend infrastructure (Node-RED/Mosquitto) and opening port 1883 on the VPN interface.

Edge Routing (Sicily): Creating a private network to connect Edge and backend without exposing MQTT to the Internet

EDGE / INTERSECTION

Smart Intersection & Local ESP32 Safety

ESP32 & Wokwi Simulator

HIL Environment: Fully virtualized Hardware-in-the-Loop simulation on Wokwi.

Core Logic: C++ firmware governed by a **Finite State Machine (FSM)**.

Asynchronous Multitasking: Abandoning blocking *delays* in favor of timers based on **millis()**.

Edge Autonomy: Traffic light cycle and safety guaranteed even in case of central backend disconnection.

Simulated Hardware Components

The barrier and road sensors are fully reproduced:

- **Traffic Light LEDs:** Red, Yellow and Green for flow control.
- **HC-SR04 Sensor:** Ultrasonics for obstacle/queue detection.
- **Servomotor:** Simulates the physical movement of the road barrier.

01 **Obstacle Detection**
HC-SR04 Sensor

→

02 **Logica di Sicurezza**
Elaborazione Local ESP32

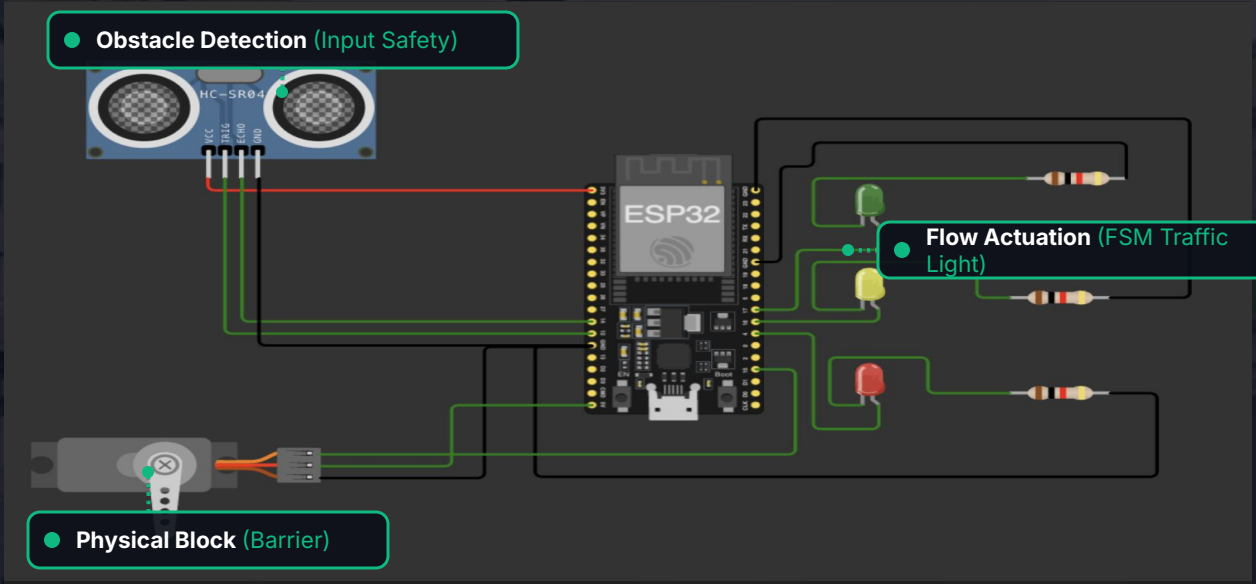
→

03 **Local Actuation**
LED & Servomotor

→

04 **MQTT Status**
Node-RED Feedback

The Edge Node: Perception and Actuation



Smart City Architecture: The microcontroller guarantees **local operational continuity** even in the event of temporary network disconnection.

EDGE DEVICE ESP32 – SENSOR DETAIL

HC-SR04 Ultrasonic Sensor & Edge Decision

Operating Principle

- **Time-of-Flight Technology:** Measures distance by calculating the flight time of the reflected ultrasonic pulse.
- **Edge-Native Processing:** Hardware data does not transit to the Cloud, but is processed instantly at a local level.
- **Specific Target:** Proximity sensor used in the prototype to simulate pedestrian detection in the critical area.

Local Decision Flow

```
Phase 1: Approach (< 300m)
  ↳ Green Wave request received
  ↳ Safety Check (HC-SR04):
    ├─ PEDESTRIAN DETECTED → Safety Abort (Red)
    └─ NO PEDESTRIAN → Activate Green Wave (Green)
Phase 2: Departure (Transit Completed)
  ↳ Safety override release
  ↳ Restoration of normal State Machine (FSM)
```

01 — **Pulse Trigger**
Ultrasonic Emission

→

02 — **Time Measurement**
Return Calculation (Echo)

→

03 — **Safety Verification**
Obstacle Presence Analysis

→

04 — **Edge Actuation**
Green or Immediate Block

Edge Computing Paradigm: The final safety decision **does not depend on the backend** , ensuring immediate responsiveness even offline.

SAFETY LOGIC (SAFETY PRIORITY)

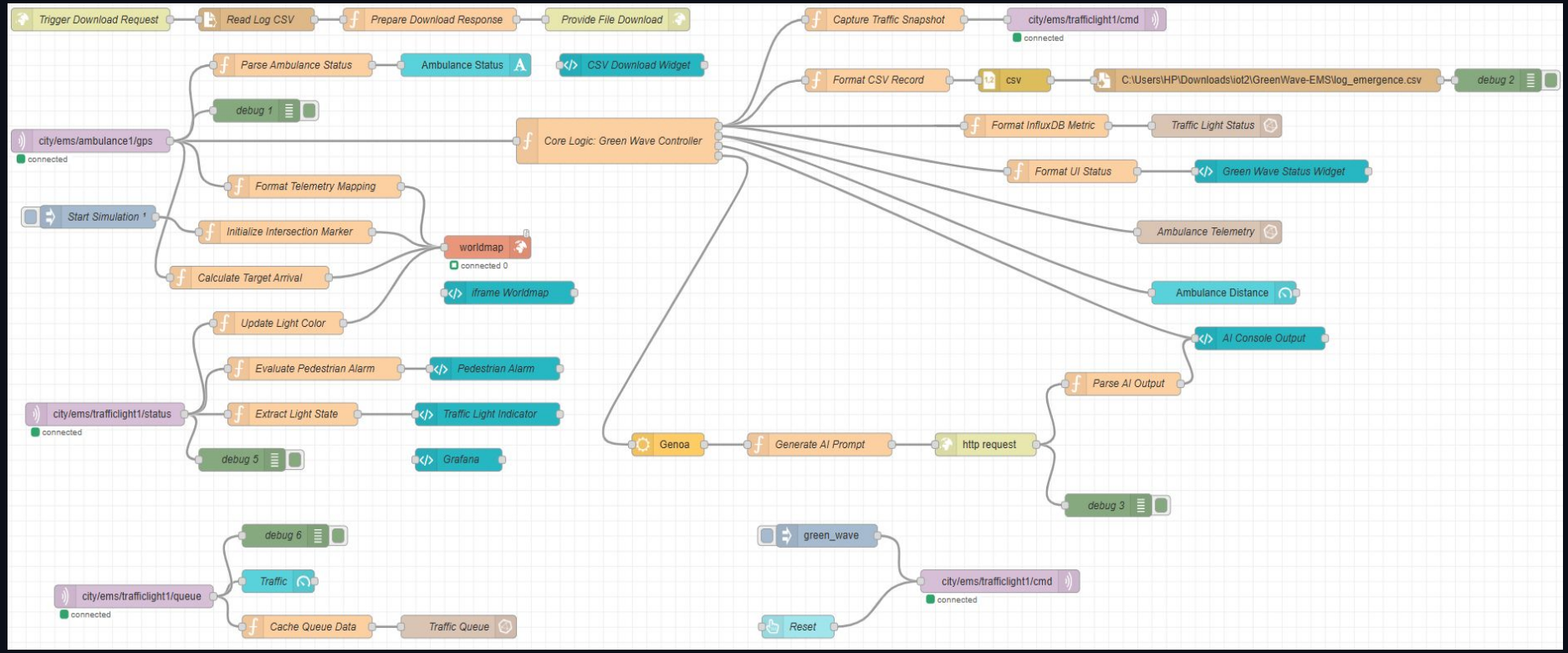
1. Green Wave Activation (< 300m)
Vehicle in critical range. If the sensor detects a clear intersection, the ESP32 immediately grants the green light.

2. Safety Abort (Override)
Pedestrian detected. The Edge ignores the Central Backend request and forces a red light, giving absolute priority to life safety.

3. FSM Reset (Post-Transit)
Ambulance passed. The ESP32 deactivates the emergency and restores the normal State Machine to allow traffic to flow.

The ESP32 safety logic acts as a **definitive local fail-safe**, overriding any central automation in the presence of obstacles.

Node-RED


Trigger Download Request
Read Log CSV
Prepare Download Response
Provide File Download
Parse Ambulance Status
Ambulance Status
CSV Download Widget
debug 1
city/ems/ambulance1/gps
connected
Core Logic: Green Wave Controller
Format Telemetry Mapping
Start Simulation 1
Initialize Intersection Marker
worldmap
connected 0
Calculate Target Arrival
iframe Worldmap
Update Light Color
Evaluate Pedestrian Alarm
Pedestrian Alarm
city/ems/trafficlight1/status
connected
Extract Light State
Traffic Light Indicator
debug 5
Grafana
Capture Traffic Snapshot
city/ems/trafficlight1/cmd
connected
Format CSV Record
csv
C:\Users\HP\Downloads\iot2\GreenWave-EMS\log_emergence.csv
debug 2
Format InfluxDB Metric
Traffic Light Status
Format UI Status
Green Wave Status Widget
Ambulance Telemetry
Ambulance Distance
AI Console Output
Parse AI Output
Genoa
Generate AI Prompt
http request
debug 3
debug 6
Traffic
city/ems/trafficlight1/queue
connected
Cache Queue Data
Traffic Queue
green_wave
city/ems/trafficlight1/cmd
connected
Reset

Telemetry Management and Green Wave Activation

1. Reception & Processing

The main flow starts from the MQTT message:

`city/ems/ambulance1/gps`

- Real-time position updates
- Map visualization and event logging
- Advanced analytics of vehicle state
- Monitoring of intervention threshold

2. Distance & Threshold Calculation

The distance from the intersection determines the behavior of the local control logic:

01 Ambulance Position + Intersection

02 Real-time Distance Calculation

03 Comparison with Preemption Threshold

Green Wave Automation and Queue Management

1. Automatic Flow

The Green Wave is deterministic, driven exclusively by telemetry.

01

GPS Detection

Ambulance sends position

02

Distance Calculation

Threshold verification on Core Logic

03

Edge Actuation

MQTT command to ESP32

2. Life Cycle & Reset

Prevention of indefinite emergency state lock.

→

Normal State

→

Emergency Detected

→

Crossing & Intersection Reset

→

Operations Restored

3. Traffic Queue Management

Analysis of Green Wave impact on ordinary vehicle flows via Node-RED.

`city/ems/trafficlight1/queue`

- **Queue & Memory:** Local history
- **HTTP Endpoint:** Impact analysis

Traffic Reaction and System Adaptation

FSM BASE

1. Ordinary Traffic

In the absence of emergencies, vehicles accumulate and flow following normal traffic light cycles. The system monitors the level of congestion.

GREEN WAVE

2. Preventive Outflow

Forcing the green light not only facilitates emergency services: it serves to **clear the intersection**, clearing the civilian traffic accumulated in front of the ambulance.

ADAPTIVE BALANCING

3. Traffic Impact Monitoring

The system guarantees vital priority to emergency vehicles, while minimizing the risk of critical congestion (gridlock) on Smart City roads.

Dashboard - Control Room

Dashboard emergence

Vehicle Telemetry

Ambulance Distance

802 m

0 1089

Ambulance Status: EMERGENCY IN PROGRESS

DOWNLOAD EMERGENCY LOG

OPEN GRAFANA HISTORY

Intersection Status

Cars in queue

40 cars

0 40

Green Wave: INACTIVE

Traffic Light Color

Pedestrian Alarm

RESET

AI Operational Report

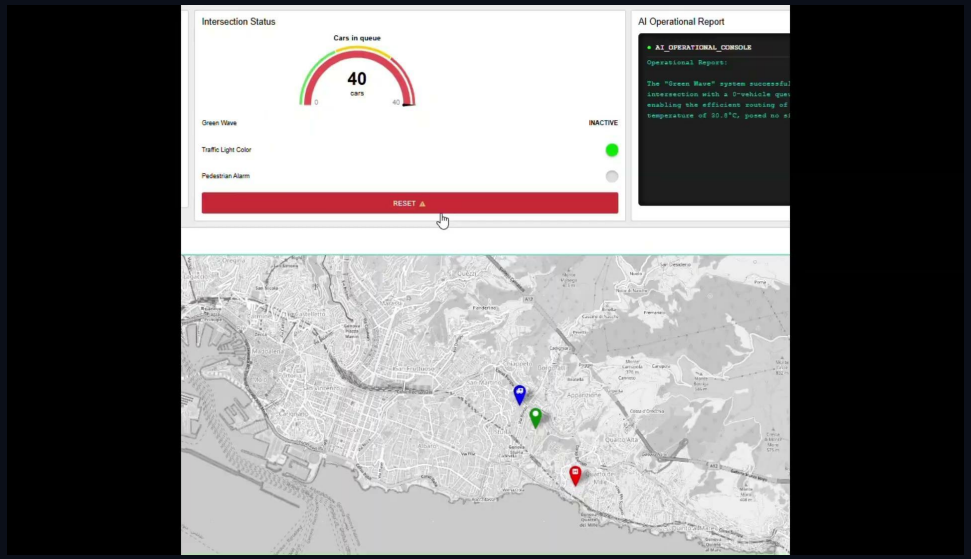
AI_OPERATIONAL_CONSOLE

Operational Report:

The successful completion of the emergency response by EMS-1 is attributed to the effective implementation of the "Green Wave" system, which enabled the ambulance to bypass the congestion caused by 8 waiting vehicles, thus facilitating efficient traffic flow. The "Green Wave" system's adaptive routing strategy allowed for a smooth and timely dispatch of the ambulance, ensuring the quickest possible response to the emergency. The current cloudy weather conditions with a temperature of 30.7°C are a contextual factor that did not impact the operation's success.

Tracking GPS

Worldmap Integration



1. Worldmap & Intersection Integration

"Real-time map rendering. The marker dynamically changes color (Green/Red) to reflect the physical state of the traffic light

2. Flashing Lights Animation

The moving ambulance icon visually simulates the activation of the flashing lights, confirming the active emergency state.

3. Target Alerting (Destination Point)

The destination indicator pulses cyclically between red shades, focusing the operator's attention on the finish line

Tactical Control: The interactive map unifies telemetry, traffic light status, and emergency events into a single geospatial interface.

GESTIONE ECCEZIONI E CONTROLLO OPERATIVO

Controllo di Sicurezza e Gestione Anomalie

1. Sicurezza Vita vs. Priorità Mezzo

L'architettura prevede un **Absolute Override**. La richiesta di precedenza dell'ambulanza viene intercettata e messa in pausa dall'algoritmo se i sensori rilevano l'ingombro critico di un pedone, ponendo la sicurezza umana al vertice della gerarchia decisionale.

2. Aggiornamento UI a Cascata

Il sistema garantisce totale trasparenza visiva all'operatore di Control Room. L'eccezione del pedone forza istantaneamente l'indicatore semaforico sul rosso e commuta lo stato dell'Onda Verde in **SOSPESO (PEDONE)**, per poi ripristinarsi in automatico.

3. Acquisizione Metriche (Gauge Traffico)

Il monitoraggio dei veicoli in coda non è solo visivo. Nel momento esatto dell'attraversamento, Node-RED esegue uno **snapshot** (congelamento) del dato; questa metrica reale diventa l'input fondamentale iniettato nel prompt per la refertazione IA.

4. Human-in-the-Loop (Reset Manuale)

Il sistema è Fail-Safe. Un pulsante di emergenza permette all'operatore di eseguire un override manuale (**state clearing**), resettando le variabili globali in caso di falsi positivi dei sensori e restituendo il controllo standard.

Controllo Tattico: Le regole di **gestione delle eccezioni** garantiscono una gerarchia decisionale immutabile dove la tutela della vita umana prevale sull'efficienza della priorità veicolare.

Data Download & History

1. Silent Logging

Automatic backup. At the end of each individual ambulance trip, Node-RED formats the structured data and automatically generates a backup on the server, ensuring history integrity.

2. On-Demand CSV Export

Operator interface. The user maintains full control: by clicking the appropriate button directly from the Dashboard, it is possible to force the retrieval of the CSV file for local offline analysis.

CONSOLE LOGS / TELEMETRY

06/09/2026	17:59:12	Green Wave Activation	Intersection Locked (Ambulance Priority)	0	287
06/09/2026	17:59:30	Emergency End	Normal Cycle Restoration	-	328
06/09/2026	18:00:20	Green Wave Activation	Intersection Locked (Ambulance Priority)	0	287
06/09/2026	18:00:37	Emergency End	Normal Cycle Restoration	-	328

LLM Integration - Contextual Analysis & Reporting

1

JSON & Prompting

Node-RED Orchestration

Node-RED collects emergency telemetry and formats it into a **structured JSON payload**. The package becomes the injected context to query Llama 3.2 via Ollama.

2

Environmental Data

Weather API Integration

The system expands road analysis. Via **OpenWeatherMap**, it retrieves real-time weather conditions, providing the AI with crucial parameters such as rain or low visibility.

3

Operational Report

AI Log Generation

At the end of the Green Wave, the language model crosses technical and weather data, independently drafting an **operational report** in natural language, ready for archiving.

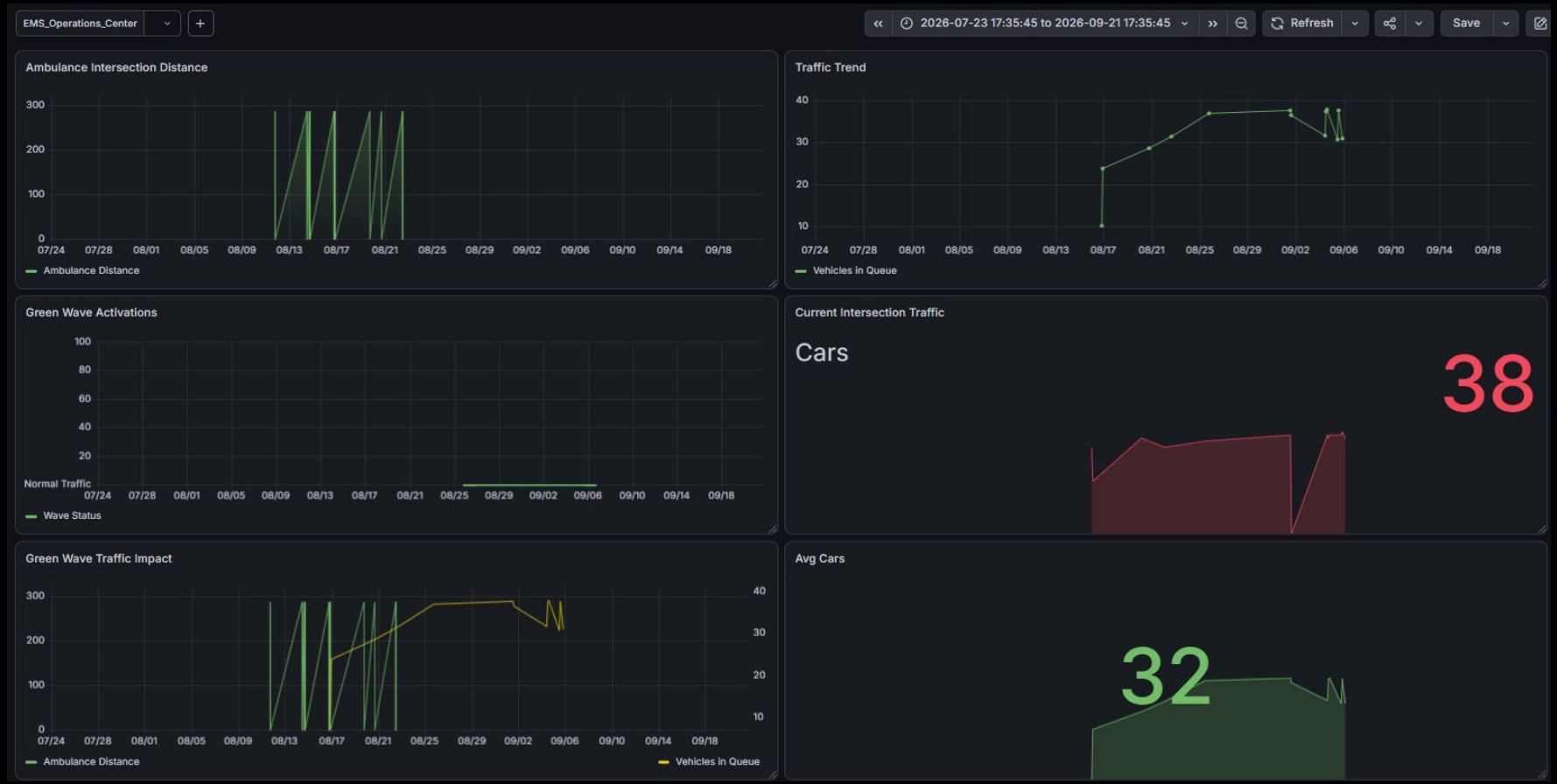
CONSOLE LOGS / AI REPORT GENERATION

```
I_OPERATIONAL_CONSOLE
rational Report:

 successful completion of EMS-1's emergency response was facilitated by the
ective implementation of the "Green Wave" system, which enabled the ambulance to
ass the congestion caused by 40 waiting vehicles, thus minimizing delay and ensu
 efficient transportation of patients to the hospital. This successful execution
 system is a testament to the thorough planning and coordination between traffic
trol and emergency services. The clear weather conditions, with a temperature of
4°C, provided optimal conditions for the traffic flow, allowing for the swift
olution of the incident.
```

AI Operational Report

```
● AI_OPERATIONAL_CONSOLE
Operational Report:

The successful completion of the emergency response by EMS-1 is attributed to the
effective implementation of the "Green Wave" system, which enabled the ambulance to
bypass the congestion caused by 8 waiting vehicles, thus facilitating efficient traffic
flow. The "Green Wave" system's adaptive routing strategy allowed for a smooth and
timely dispatch of the ambulance, ensuring the quickest possible response to the
emergency. The current cloudy weather conditions with a temperature of 30.7°C are a
contextual factor that did not impact the operation's success.
```

Grafana - Operations Center



Advantages of the Hybrid Architecture

Simulation
Wokwi

Development and testing of ESP32 without the need for physical hardware.

Modularity
MQTT

Replacement or modification of devices without redesigning the system.

Security
Local

ESP32 maintains safe decisions even in case of backend downtime and also Tailscale and its encryption.

Local AI
Ollama

Language capabilities without dependency on external cloud services.

Architectural Flow

Architecture Legend: **Red = Edge/Data Source** | **Blue = Broker & Actuators** | **Green = Orchestration & Core Logic** | **Purple = Generative AI Integration**

TECHNOLOGICAL ARCHITECTURE

Objects and Technologies Used in the System

Hardware & Edge Simulation

Hardware / Simulated Hardware

- **ESP32:** Microcontroller used as the intersection's Edge Device.
- **HC-SR04:** Ultrasonic sensor for local obstacle detection.
- **LED (Red, Yellow, Green):** Representation of traffic light signals.
- **Servomotor:** Simulation of the physical safety barrier.

Development Environment

- **Wokwi:** Environment used to develop, test, and simulate the ESP32-based smart intersection.

Software, Protocols & Services

Languages & Runtimes

- **Node.js:** Used to simulate the mobile Edge device (ambulance).
- **JavaScript & C++:** Simulator logic, Node-RED nodes, and ESP32 firmware.

Networking & Security

- **Tailscale:** Zero-Trust VPN network for secure connection of distributed nodes.

Communication & Integration

- **MQTT:** Protocol and broker (intermediary) used for asynchronous Pub/Sub messaging between hardware and server.
- **Node-RED:** Central orchestrator and stream processor of the backend.

Visualization & Artificial Intelligence

- **InfluxDB:** Time-Series Database (TSDB) used for persistent storage of high-frequency telemetry.
- **Grafana & Worldmap:** Historical monitoring and GPS geographical tracking.
- **Ollama & LLM:** Local generation of the post-emergency operational report.
- **OpenWeatherMap:** Real-time weather data.

The Four Architectural Levels of the Project

Level 1

Perception

Data acquisition and environment state:

- Simulated GPS
- HC-SR04 Sensor
- Ambulance status
- Traffic detection

Level 2

Transport

Communication and messaging infrastructure:

- MQTT Protocol
- Central broker
- Dedicated topics
- Tailscale

Level 3

Processing

Central logic and data processing:

- Node-RED
- InfluxDB
- Distance calculation
- Green Wave
- Queue management
- Traffic light status
- Logging & data prep
- Ollama & LLM

Level 4

Application

Operator interface and monitoring:

- Operational dashboard
- Worldmap map
- Grafana
- Status visualization
- CSV Export
- Operational report

EDGE & ARCHITETTURA LOCALE

Componenti e Logica del Sistema (Edge-side)

ESP32

Firmware C++ (FSM)

Gestisce i cicli semaforici e l'attuazione fisica in totale autonomia locale.

Sensore HC-SR04

Time-of-Flight

Rilevamento ostacoli. Innesca il blocco d'emergenza (Safety Abort) ignorando il Cloud.

Node.js

Digital Twin

Digital Twin del mezzo di soccorso. Calcola l'avvicinamento (Haversine) e pubblica la telemetria JSON.

Wokwi & VS Code

Simulatore HIL

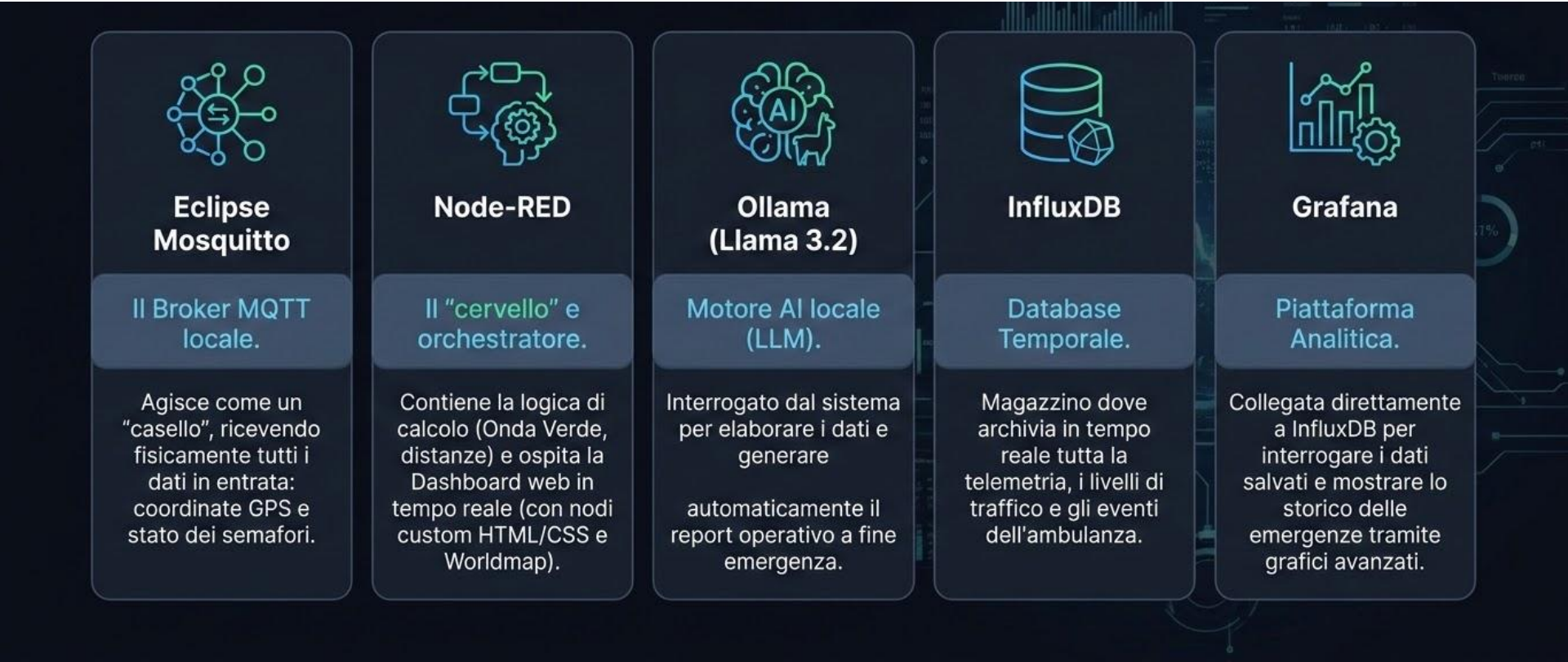
Permette il test intensivo e la validazione di codice e circuiti in locale.

Tailscale VPN

Tunnel Crittografato

Connette l'Edge al Backend isolando il traffico dall'internet pubblico.

Componenti backend control



FUTURE PERSPECTIVES

Future Developments: Towards an Advanced Smart City

1. GNSS RTK & Sensor Fusion

Centimeter-level localization.

Transition to GNSS RTK receivers for 1-2 cm accuracy. Integration with inertial sensors (IMU) ensures smooth distance calculation in urban canyons.

2. Network Expansion

More intersections & vehicles.

Scale the project to cover an entire metropolitan area, extending the service to other emergency categories such as police and fire departments.

3. AI & Predictive Analysis

Advanced tactical support.

ML algorithms predict congestion and optimize routes, while LLM acts as a virtual assistant to query the system in natural language.

Thank you for your attention! GreenWave EMS

THE DEVELOPMENT TEAM

The project was entirely conceived, programmed, and simulated by:

Alessio Scarpulla & Giorgio Innocenti